

Uniform Call Syntax for explicit-object member functions

Document #: P2818R0
Date: 2023-03-15
Project: Programming Language C++
Audience: EWG
Reply-to: Gašper Ažman
<gasper.azman@gmail.com>

Contents

- 1 Introduction
- 2 Motivation and Prior Art
 - 2.1 Related Work
- 3 Proposal
 - 3.1 Alternatives to syntax
- 4 Ruminations from certain past EWG chairs on UFCS design feasibility
- 5 How is this different from prior art?
 - 5.1 There can be no confusion about which function is preferred
 - 5.2 It's precise
 - 5.3 It's simple and minimal
 - 5.4 It's modular
- 6 Future Extensions
 - 6.1 Ruminations of certain past EWG chairs on the design of the extension
 - 6.2 Use in the standard library
- 7 Questions for EWG
- 8 FAQ
 - 8.1 Why are you writing another paper about UFCS?
 - 8.2 Has this been implemented?
 - 8.3 Are you going to bring the extension methods paper too?
 - 8.4 Can I put this not on the first argument?

- [8.5 How do I define this friend thing out-of-line?](#)
- [8.6 Can these be private?](#)
- [8.7 Should we just make this the default?](#)
- [8.8 How is this better than a forwarding hidden-friend function?](#)
- [9 References](#)

§ 1 Introduction

This paper introduces a unification of hidden friends and explicit-object member functions to allow a limited, but hopefully uncontroversial Uniform Call Syntax for them.

Unlike the previous proposals on this topic, this one avoids pretty much all controversy.

§ 2 Motivation and Prior Art

Why we might want to have UFCS in the language has been covered extensively already, and Barry Revzin classified all approaches and issues in [\[Revz\]](#).

The short summary boils down to a few points:

- generic code wants to use free-function notation because it's more general
- users with IDEs want to use member-function call notation because it's easier for autocomplete
- it makes it possible to write idiomatic code in more contexts because one has more choices.

While the above list may be a bit short, at least the first point is *so incredibly important* that the list of papers on the subject is staggering.

Note: the **CS:** and **OR:** labels refer to semantic options in the above-linked article. Please read it, it's short.

The post very helpfully lists much prior art by many of WG21's esteemed members: Glassborow, Sutter, Stroustrup, Coe, Orr, and Maurer; specifically [\[N1585\]](#), [\[N4165\]](#), [\[N4174\]](#), [\[N4474\]](#), [\[P0079R0\]](#), [\[P0131R0\]](#), [\[P0251R0\]](#), [\[P0301R0\]](#) and [\[P0301R1\]](#).

With regards to the taxonomy proposed in [\[Revz\]](#), this paper is sortish in the CS:FreeFindsMember category, but with CS:MemberFindsFree and CS:ExtensionMethods left as a possible future extensions, as they aren't mutually exclusive.

This paper proposes OR:OneRound, but with ambiguities being impossible (ill-formed) due to the way this is done.

§ 2.1 Related Work

There have been thoughts around `operator.FQN()`, Kirk Shoop and Ville Voutilainen seem to have a reasonable grasp of how far that got.

The pipeline operator in its various incarnations also is roughly related - the people at the head of that one seem to be Barry Revzin, Colby Pike [P2011R1] and Isabella Muerte [P1282R0].

§ 3 Proposal

We propose that marking an explicit-object member function as a `friend` (to parrot inline friend function declarations, specifically hidden friends) would also make it callable via free-function argument-dependent-lookup.

Example:

```
struct S {
    friend int f(this S) {
        return 42;
    }
};

int g() {
    S{}.f(); // OK, returns 42
    f(S{}); // OK, same
    (f)(S{}); // Error; f can only be found by ADL
}

// note: forward declaration
int f(S); // Error, int f(S) conflicts with S::f(S)
int f(S) {}; // Error, int f(S) conflicts with S::f(S)

int f(int); // OK
```

That's pretty much it.

§ 3.1 Alternatives to syntax

While `friend` communicates exactly what this actually does, it only does so if one knows about hidden friends.

We *could* use a context-sensitive keyword here, like `adl`, or `associated(type-id-or-class-template-id, ...)`, as Lewis Baker's paper [D2823R0] proposes, and just say that these functions are callable via ADL for the classes mentioned in the parentheses using this free-function notation. We still need this paper's wording to get that accomplished.

The main issue with `associated(type-id-or-class-template-id, ...)` is that type-ids cannot be dependent, as that would be equivalent to adding the overload to every namespace.

`friend` does not introduce this complexity, but the additional power might be useful enough to choose as preferred syntax for the semantics this paper proposes.

§ 4 Ruminations from certain past EWG chairs on UFCS design feasibility

(very slightly paraphrased)

In general, concerns arise from the existing guarantee that member functions simply don't mix with ADL. Using a member call syntax means ADL doesn't happen, but declaring a function a member also makes it not dance with ADL overloads in an overload set. The first guarantee is paramount, the second is Really Nice To Have.

Granted, the proposal still allows having the second guarantee. Don't mark your functions `adl, done`.

§ 5 How is this different from prior art?

§ 5.1 There can be no confusion about which function is preferred

There is only one function in the first place.

The `friend` syntax signals the behavior exactly. The declaration of the member function is *also* injected into the type's "hidden" namespace as a hidden friend after notionally removing the keyword `this` from the argument list.

This is OK, because explicit-object member functions have free-function type, and their bodies behave as if they were free functions, so we're not lying. We're doing exactly what it looks like.

§ 5.2 It's precise

You opt-in to UFCS on a per-declaration basis. This matters because UFCS is primarily about enabling generic code to use a given type, and gives precise control about both the free-function and member-function interfaces of a given class. When both interfaces should provide a given signature, this is the only proposal that lets you *just do that and only that*, without impacting other parts of either overload set.

§ 5.3 It's simple and minimal

It just merges two things we already have - hidden friends, and explicit-object member functions. No need to remember which comes first or how a given function is defined - both syntaxes always dispatch to the *only* implementation.

§ 5.4 It's modular

It does not propose, but does not preclude, future extensions into, well, extension methods. See the Future Extensions chapter.

§ 6 Future Extensions

While the author of this proposal is of a mild opinion that Extension Methods (CS:ExtensionMethods) would not carry their weight in C++, this paper is specifically neutral on this topic and reserves the only plausible syntax for them:

```
// Disclaimer: NOT PROPOSED IN THIS PAPER
struct E {};
int h(this E) { return 42; } // look ma, I'm not a member of E
int main() {
    h(E{}); // ok, obviously, since h is declared outside of E
    E{}.h(); // also OK, h found by ADL and specifies where to put `this`.
}
```

There is one caveat - in this case, if E declares an h(this E), it would conflict at declaration time, since this proposal already specifies that behavior.

§ 6.1 Ruminations of certain past EWG chairs on the design of the extension

(very slightly paraphrased)

We can do a double-requirement. The extension needs to be declared with an explicit object parameter **and** the class needs to say that it allows such extending. Then we're good with the extension too.

§ 6.2 Use in the standard library

Consider the ranged for loop and the magic it does with looking for begin and end.

If we had *had* this facility from the beginning, we probably *wouldn't have invented that magic*, but just added friend to begin() and end() member functions of all standard containers instead, so that the free-function version was accessible in all contexts.

std::ranges might have had a substantially different design if [\[P0847R7\]](#) and this facility had been available. Note that std::ranges::begin looks at free function *and* members to decide which one to call.

§ 7 Questions for EWG

1. are we OK choosing the *OR:OneRound* (+no conflicting declarations) approach, knowing that it eliminates *OR:TwoRoundsPreferAsWritten*, *OR:TwoRoundsMemberFirst* and *OR:OneRoundPreferMembers* for all UFCS-related features in the language?
2. Do we want a different syntax from *friend* to signal exactly what *friend* does in this context, like a context-sensitive keyword `associated(type-id, ...)`?
3. Do we find UFCS eliminates a significant-enough portion of library boilerplate in the cases where a class needs to provide both interfaces for this feature to be worth the implementation cost?

§ 8 FAQ

§ 8.1 Why are you writing another paper about UFCS?

Because this is a novel direction that might actually fit the language and pass.

§ 8.2 Has this been implemented?

No, but given that it uses a syntax that is ill-formed in C++23, and that it only inserts an alias to the same function that otherwise works exactly like a hidden friend, I *really* don't have implementation concerns. Any compiler that implements [\[P0847R7\]](#) will have zero issues implementing this paper.

§ 8.3 Are you going to bring the extension methods paper too?

No. I don't need them, and injecting functions into the space of member functions that is explicitly given to the class designer is wrong unless properly scoped. I don't know how to properly scope it. If you do, the only reasonable syntax is above.

§ 8.4 Can I put this not on the first argument?

Not yet. I might bring that paper if this one passes, but separately.

§ 8.5 How do I define this friend thing out-of-line?

It's still an explicit-object member function, so normally:

```
struct S {  
    friend int f(this S);  
};
```

```
int S::f(this S) { return 42; }
```

§ 8.6 Can these be private?

No, friends can't be private, and so this can't either.

Fortunately, making it private has no reasonable use - if you're in the class implementation where you can use private methods, you're not in a generic context where you'd have to use free-function syntax.

§ 8.7 Should we just make this the default?

No. Most of the time, you won't want to make your explicit-object member functions also public. This has to do with satisfying concept requirements, not getting closer to general UFCS.

§ 8.8 How is this better than a forwarding hidden-friend function?

§ 8.8.1 Forwarders can't correctly forward prvalues

Any time you decay something to a reference in a forwarding function, the compiler has to ODR-use the move-constructor.

This matters for immovable types:

```
struct immovable {
    immovable() = default;
    immovable(immovable&&) = delete;
};

void takes_immovable(immovable x) {}
auto forwarding_function(auto&& x) {
    return takes_immovable(std::forward<decltype(x)>(x));
}

auto h() {
    takes_immovable(immovable{}); // OK, direct construction

    // Error, use of deleted function immovable(immovable&&)
    forwarding_function(immovable{});
}
```

In general, forwarding functions don't work for immovable types, and `std::execution` and related things will generate *a lot* of them.

The work-around is to use lazy construction such as with `in_place` (see below) but that generates a lot of bloat the optimizer has to remove and is far from free in terms of syntax.

This *especially* matters for construction of aggregates that have immovable members, since they don't (and can't) have user-declared constructors, although this problem isn't tackled by this paper.

With this paper, we declare *true* aliases to member functions, so the following will work:

```
struct Logger {  
    // OK, log(logger, immovable{}) and logger.log(immovable{}) both work  
    friend void log(this Logger&, auto... args);  
  
    friend void log_free(Logger& self, auto&&...args) {  
        // does not work with immovable types  
        self.log(std::forward<decltype(args)>(args)...);  
    }  
};
```

§ 8.8.2 Addendum: in_place

```
template <typename F>  
struct in_place {  
    F_ f;  
  
    operator decltype(std::move(f)()) && {  
        return std::move(f)();  
    }  
};  
template <typename F>  
in_place(F) -> in_place<F>;  
  
// usage  
forwarding_function(in_place([]{return immovable{};})); // works, but  
// breaks overload resolution in case of templates  
// void takes_immovable(std::same_as<immovable> auto x) is broken because  
// it doesn't force a conversion
```

§ 9 References

[D2823R0] Lewis Baker. Declaring hidden non-friend functions to be found by argument-dependent-lookup.

<https://wg21.link/D2823R0>

[N1585] Francis Glassborow. 2004-02-05. Uniform Calling Syntax (Re-opening public interfaces).

<https://wg21.link/n1585>

[N4165] Herb Sutter. 2014-10-04. Unified Call Syntax.

<https://wg21.link/n4165>

[N4174] Bjarne Stroustrup. 2014-10-11. Call syntax: x.f(y) vs. f(x,y).

<https://wg21.link/n4174>

[N4474] Bjarne Stroustrup, Herb Sutter. 2015-04-12. Unified Call Syntax: x.f(y) and f(x,y).

<https://wg21.link/n4474>

- [P0079R0] Roger Orr. 2015-09-28. Extension methods in C++.
<https://wg21.link/p0079r0>
- [P0131R0] Bjarne Stroustrup. 2015-09-27. Unified call syntax concerns.
<https://wg21.link/p0131r0>
- [P0251R0] Bjarne Stroustrup, Herb Sutter. 2016-02-11. Unified Call Syntax Wording.
<https://wg21.link/p0251r0>
- [P0301R0] Jens Maurer. 2016-03-04. Wording for Unified Call Syntax.
<https://wg21.link/p0301r0>
- [P0301R1] Jens Maurer. 2016-03-21. Wording for Unified Call Syntax (revision 1).
<https://wg21.link/p0301r1>
- [P0847R7] Barry Revzin, Gašper Ažman, Sy Brand, Ben Deane. 2021-07-14. Deducing this.
<https://wg21.link/p0847r7>
- [P1282R0] Isabella Muerte. 2018-09-27. Ceci N'est Pas Une Pipe: Adding a workflow operator to C++.
<https://wg21.link/p1282r0>
- [P2011R1] Barry Revzin, Colby Pike. 2020-04-16. A pipeline-rewrite operator.
<https://wg21.link/p2011r1>
- [Revz] Barry Revzin. What is unified function call syntax anyway?
<https://brevzin.github.io/c++/2019/04/13/ufcs-history/>